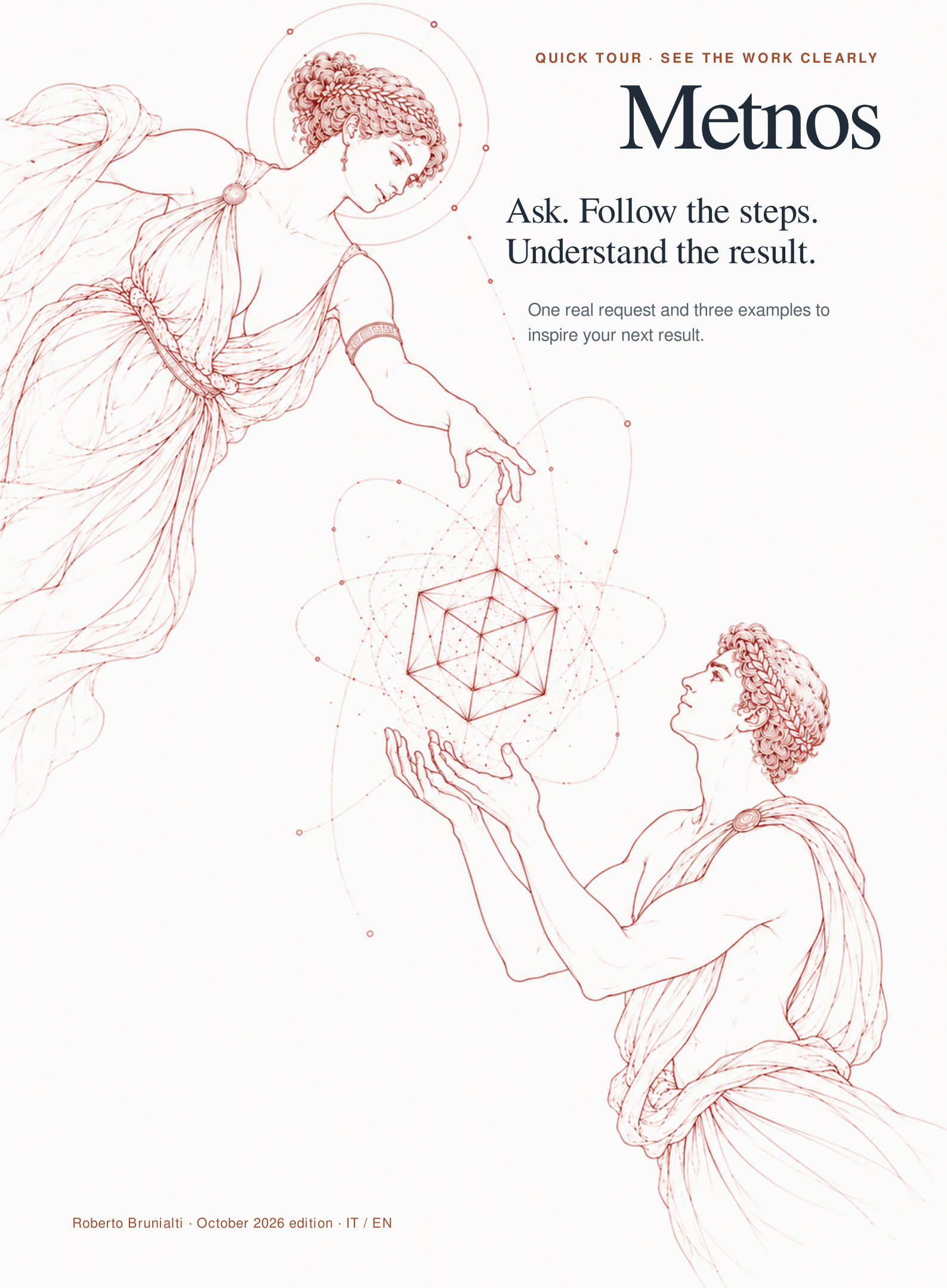
QUICK TOUR · SEE THE WORK CLEARLY

Metnos

Ask. Follow the steps. Understand the result.

One real request and three examples to inspire your next result.

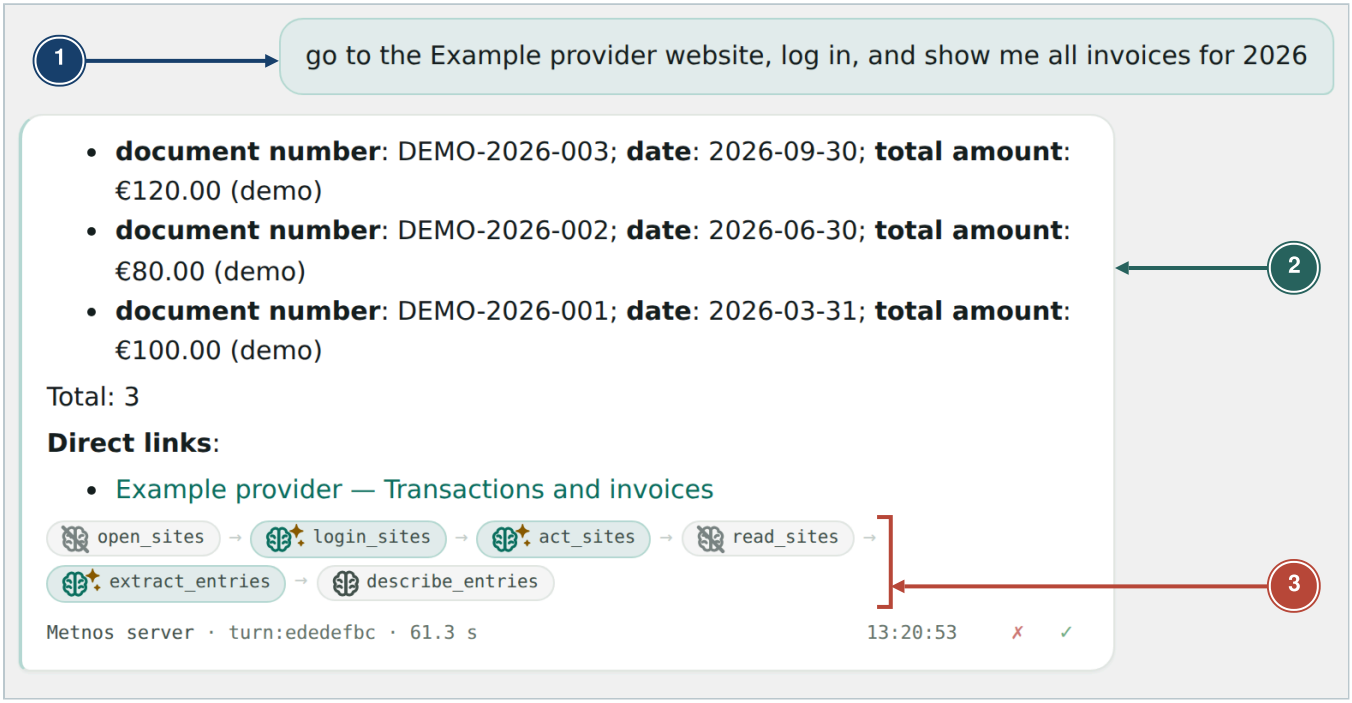


Roberto Brunialti · October 2026 edition · IT / EN

01 A REAL CASE

One request. Six executors at work.

From logging into the portal to listing the invoices: one request coordinates six functions and shows the result.



Based on a successful request, translated into English. Names, invoice numbers, amounts and addresses use sample data. Open the image to zoom in.

1 Request

A goal in everyday language: log into the portal and find the 2026 invoices.

2 Result

Three invoices with a number, date and amount. A link takes you back to the source.

3 Steps

Six executors show how Metnos reaches the result, from logging in to presenting the data.

Request completed successfully. Access to the portal, three invoices for 2026 listed with their details, and a link to the original section.

02 READ THE SCREEN

Every mark has a meaning.

The trace shows what was invoked. The final row identifies the turn so you can find it again in Activity → Turns.

Number, date, amount

Each invoice has a number, date and amount. The date 2026-09-30 means 30 September 2026. The visible numbers and amounts are sample data.

Total: 3

This is the number of invoices listed, not the sum of their amounts: one from March, one from June and one from September.

Direct links

“Transactions and invoices” links back to the portal documents, so you can check the result against its source.



Trace from the real turn: the technical names identify the invoked functions, not six different models.

1 Names, arrows and colours

`open_sites → login_sites → act_sites → read_sites → extract_entries → describe_entries`: the order of steps. Each name identifies a function; arrows and colours help you follow the sequence.

2 Brain and $ symbols

DET: opening and reading. **AGENT**: logging in, navigating and extracting. **LLM**: presenting the data. No **$** appears here: the steps shown report no cloud model calls.



Final row from the real turn. Duration and time of day are different measures.

3 Execution location

Metnos server is the destination shown for this turn. A paired device is shown by its name.

4 Turn and duration

`ededefbc` is the first eight characters of the turn identifier. **61.3 s** is the total turn duration, not just model time.

5 Time and feedback

13:20:53 is the completion time shown in the chat. ✗ / ✓ let you rate the response; they are not certificates of success.

A result you can check. Three invoices found, with a link to the source. This case shows document lookup.

03 TYPES, ORIGINS AND COSTS

How it works. Where it comes from.

The brain identifies **the executor type**, not the number of model calls. A step may take a path that uses no model. All three icons appear in the real case.

DET · Deterministic

Runs rules and code. That step does not use a model; preparing the plan may still use one.

LLM · Model assisted

Can use a language model for part of the work, such as presenting or transforming data.

AGENT · Internal decisions

Observes, chooses and acts through multiple steps within an assigned mandate and limits. It may stop before reaching the outcome.

Origin answers a different question

The executor list also shows where a component comes from. Origin, model use and execution location are separate properties.

`handcrafted`

Written and maintained in the project.

`synthesized`

Generated with a model and admitted into the system. This does not imply a model call on every run.

`imported`

Derived from an external skill, with recorded provenance and admission into the system.

`builtin`

A service built into the runtime. Some are available to the planner; others are restricted.

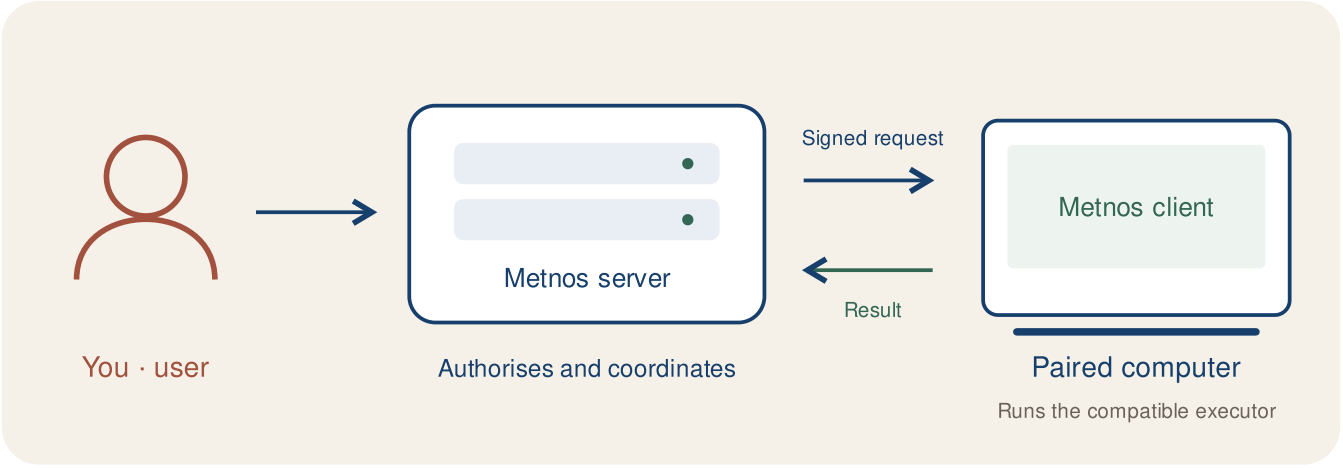
$ means cloud, not a price

The **$** symbol, when present, indicates cloud model calls reported by an executor. It does not appear in this case. It is not a price, and its absence does not make the entire turn free: the planner may also use a model. Consult Monitor for recorded call measurements.

04 USERS AND DEVICES

Your user account. On your devices, too.

Metnos can pair one or more computers with a user and run executors on those devices. The target computer needs the Metnos client.



1 Pair the device with a user

In Settings → System → Devices, the administrator selects the owner and gives the computer a name.

2 Open the link on that computer

The guided process installs the Metnos client and establishes a verifiable identity. The initial link expires after a limited time.

3 Run compatible executors there

The server sends authorised, signed requests. The client verifies the component, runs it and returns the result.

Close to your files, within granted permissions. Reachability alone is not enough: compatibility, resources and authorisation also matter. An administrator can revoke the device.

Explanatory diagram. The invoice case on the preceding pages ran on the server and does not demonstrate execution on a paired computer. Device guide →

05 AUTHORISE AND VERIFY

Actions matter. So do their boundaries.

The convenience of a single request should come with clear permissions, verifiable results and explicit limits.

1 **Request**
An outcome to achieve

→

2 **Review**
Permissions and required approvals

→

3 **Execution**
Steps within limits

→

4 **Verification**
Outcome and trace

Protected credentials

When credentials are needed, use the dedicated form with its protected password field. Do not put them in the chat's free text.

Visible limits

The response keeps a reference to the source. When errors, missing access or partial results occur, those limits must remain visible.

Approvals follow the policy

Actions go through the instance's checks. When confirmation is required, you can read the proposed operation before authorising it.

Undo where supported

Undo depends on the operation and the available records. Not every external effect is reversible.

EXAMPLE FOLLOW-UP REQUEST

"Before sending these invoices, show me who will receive them and what will be sent."

Convincing because you can check it. Read the outcome, open the result and compare it with the source before treating a sensitive task as complete.

06 MORE IDEAS, SAME CHAT

Three requests. Three possibilities.

From files to emails, one request can connect several actions. Illustrative examples from the previous guide.

A

Identical images → spreadsheet

“Count the files in this folder, find identical images and create a spreadsheet.”

The result. A count summary and a spreadsheet linking each copy to its reference, with names and paths. The originals stay in place.

Matching compares file contents rather than visual similarity.

B

Invoice emails → delivery

“Find emails containing this month’s invoice details, create a spreadsheet and ask for my confirmation before sending it.”

The result. Dates, suppliers, numbers and amounts are organized, with missing details flagged. You review the recipient and attachment before delivery.

C

Photos on your PC → undo

“On my paired PC, organize photos into folders by modification year.”

Then: “Undo the last operation.”

The result. Photos return to their original paths and newly created folders are removed when empty.

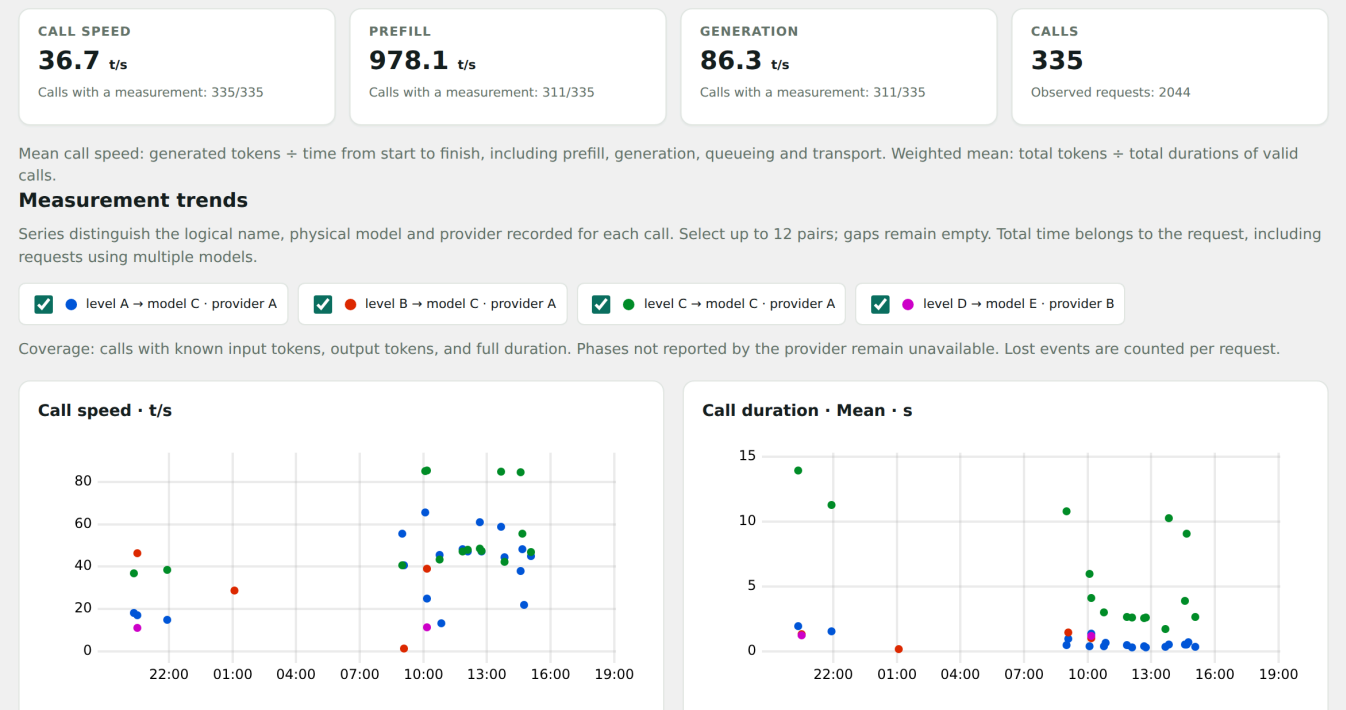
The PC must be online, with the files and paths still available.

Explore more use cases →

07 MONITOR · MODEL PERFORMANCE

See how your models perform.

Timing, speed and usage: Monitor → Model performance gives you an overview of recorded calls.



Sample names; real measurements from 24 hours of activity. Open the image to zoom in.

Compare models

The colored series show which models were used and their roles.

Understand the wait

Timing and tokens per second help explain response speed.

Put numbers in context

The period and number of calls give the measurements perspective.

From one request to the wider picture. Chat tells the story of a task; Monitor helps you understand how the system is working.

Explore Monitor →

08 GET STARTED AND GET GUIDANCE

From your first question to your own data.

Use the guided installation. Then ask the Tutor how to connect a source or use a function without having to guess technical names.

Install and resume

The installation guide walks you through setup. The preliminary check applies no changes; an interrupted procedure resumes from valid steps.

Ask the Tutor

The Tutor explains Metnos using public documentation. A “how do I?” question provides guidance; asking to perform an action is a separate step.

```
git clone https://github.com/brunialti/metnos.git
cd metnos
bash install/bootstrap.sh --check
bash install/bootstrap.sh
```

How do I mount a NAS folder in Metnos?

A **mount** makes a NAS share accessible as a local folder without copying its files. On Linux, Metnos can prepare an **SMB/CIFS** mount when the administrative operation is available and authorised.

> “Mount `//storage.example.org/Photos` in Metnos’s Pictures folder, read-only.”

The address and share are examples. The folder must be accessible to the **service account**. If credentials are needed, use the **protected password field**. If the NAS is already accessible, reuse the **existing path**.

Persistence after a restart requires separate configuration: a successful mount **does not guarantee** it. Check by listing the files. Connecting a folder **does not index** it: the first photo search may start preparation in the background. The Tutor explains the procedure but **does not mount anything**.

The Tutor → · Preparing photo search →

09 YOUR NEXT STEP

A request of your own. A result to understand.

Start with a simple question. When a task becomes complex, follow the trace and use the sources to verify the result.

GET ORIENTED

"How do I pair a computer with my user account?"

READ

"List the files in the Pictures folder."

VERIFY

"What is missing before this result can be considered complete?"

Explore the interface

Chat, Activity, Monitor and Settings.

Find a use case

Available functions and example requests.

Connect a device

Pairing, identity and revocation.

Understand executors

Code, models and mandates.

A concrete promise. Describe a task in natural language, follow its execution and understand what succeeded. The trace and source references help you assess the result.

11 October 2026 edition · Italiano / English
Chat and Monitor show real data in the current interface. Personal details use sample data; screenshots are translated in this English edition. The three additional scenarios and diagrams are illustrative.

Executor catalogue · Devices · Full documentation

Metnos · Roberto Brunialti · open source, MIT licence · metnos@metnos.com